

Design and Analysis of Algorithms

- *Course Code:* BCSE204L
- *Course Type:* Theory (ETH)
- *Slot:* A1+TA1 & & A2+TA2
- *Class ID:* VL2023240500901
VL2023240500902

A1+TA1

Day	Start	End
Monday	08:00	08:50
Wednesday	09:00	09:50
Friday	10:00	10:50

A2+TA2

Day	Start	End
Monday	14:00	14:50
Wednesday	15:00	15:50
Friday	16:00	16:50

Syllabus- Module 6

Module:6 | **Randomized algorithms**

5 hours

Randomized quick sort - The hiring problem - Finding the global Minimum Cut

Deterministic Algorithm

- Given a **specific input**, a deterministic algorithm will consistently produce the **same output** and follow the same set of instructions. There's **no randomness** involved.
- Due to their predictable nature, deterministic algorithms are widely studied and used as they can be efficiently executed on real machines.
- Examples include
 - Deterministic Turing Machines and
 - Deterministic Finite Automata.
- **Execution time** is **polynomial** since the outcome is consistent on different executions.

Non-Deterministic Algorithms

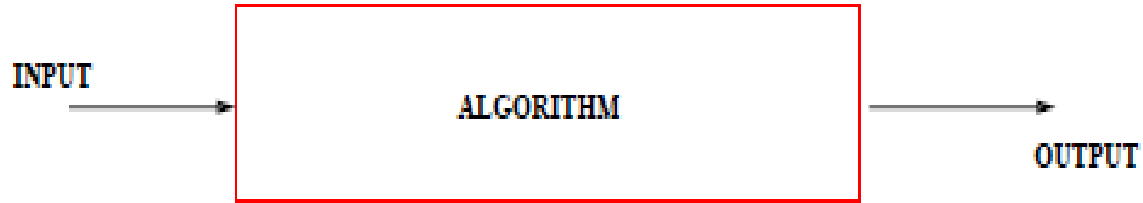
- Unlike deterministic algorithms, non-deterministic algorithms can exhibit **different behaviors (output)** even for the same input (**Unpredictable behavior**).
- Non-unique results: The outcome of a non-deterministic algorithm may not be uniquely defined, and it could involve randomness in its execution path.
- Less practical: Non-deterministic algorithms are generally not used in real-world applications due to the difficulty of implementation and analysis.
- Can't solve problems in polynomial time due to their varying behaviors.

Randomized algorithms

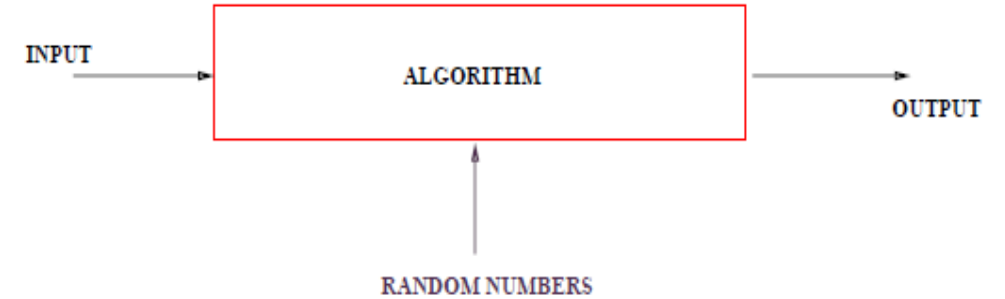
- Randomized algorithms incorporate **random numbers** (at least one) during their computations to make decisions. This "simple randomization" adds an element of chance to the process.
- Simple Randomization means **Algorithms + Randomness**.
- Randomized algorithms may produce **different outputs** or **execution times** even with the **same input**, depending on the random choices made.
- **Reason to use Randomize:** While introducing randomness, the design and analysis of these algorithms aim to ensure their **effectiveness** for various inputs.
- **Benefits:** Randomized algorithms can sometimes improve an algorithm's efficiency (time or space complexity) compared to a **purely deterministic approach**.

Randomized algorithms

Deterministic Algorithms



Randomized Algorithms

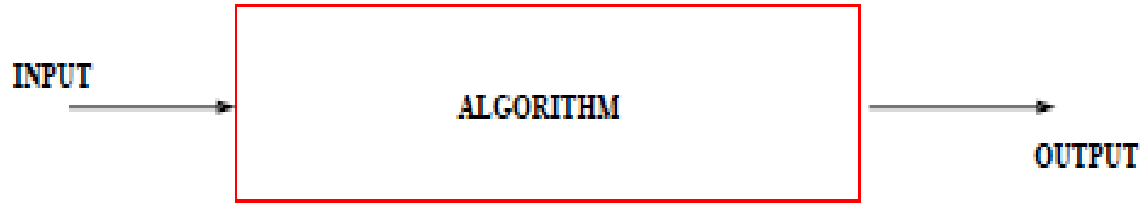


Though they use randomness, randomized algorithms are **not truly non-deterministic** because the randomness is a controlled element within the algorithm's logic.

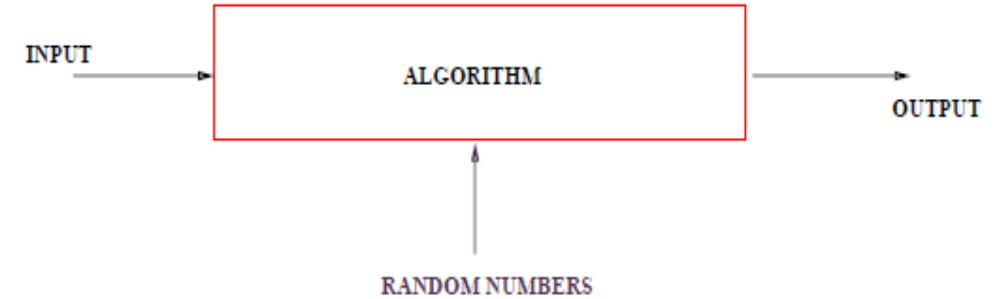
Note: Deterministic algorithms are reliable and predictable, while non-deterministic algorithms are less practical due to unpredictable behavior. Randomized algorithms strategically use randomness to potentially improve efficiency.

Randomized algorithms

Deterministic Algorithms



Randomized Algorithms



Is randomized algorithms are comes under deterministic or non-deterministic algorithms.

Randomized algorithms are neither strictly deterministic nor truly non-deterministic. They fall somewhere in between:

- Like non-deterministic algorithms, randomized algorithms can have different outputs or execution times for the same input because of the random choices they make. This element of chance introduces some unpredictability.
- Unlike non-deterministic algorithms, randomized algorithms are designed and analyzed for their behavior with randomness. The randomness is a controlled element within the algorithm's logic, not a theoretical exploration of all possible paths simultaneously.

A Simple Example: Choosing a Large Number

- Given n numbers, find a number that is $\geq$ median
- Simple deterministic algorithm takes $O(n)$ time
- A simple randomized algorithm:
 - Choose k numbers randomly, $k < n$, and output the maximum
- Runs faster
- But this may not give the correct result always
 - But the probability that an incorrect result is output is less than $\frac{1}{2^k}$
 - Somewhat trivial example, but highlights the essence of a large class of randomized algorithms

Handwritten notes: $\frac{1}{2^k}$ and $\frac{1}{2^{20}}$

Handwritten notes: 1 2 3 5 6 7, $k=3$

Handwritten notes: $n=100$, $k=20$, $\frac{1}{2^{20}}$

10	20	30	40	50	60	60	60	60	60
----	----	----	----	----	----	----	----	----	----

```
Algorithm Repeated Element(a,n)
// Finds the repeated element from a[1:n]
{
While(true) do
{
i:=Random() mod n+1;
J:=Random() mod n+1;
If((i≠j) and (a[i]=a[j])) then
return i;
}
}
```

Another Example: Randomized Quick Sort

- The classic Quick Sort algorithm chooses a pivot element and partitions the array around it. In the **worst case**, if the chosen pivot always ends up on one extreme of the array, it can lead to a **slow sorting process** (n^2 time complexity).
- In this variation, a random number is used to pick the pivot element. This randomness helps to avoid the worst-case scenario of the standard Quick Sort. On average, the randomly chosen pivot will split the array more evenly, leading to a much faster sorting process (typically **$O(n \log n)$** time complexity).

Important Note:

Using randomness to reduce the time complexity of a standard algorithm.

- **Decision Making with Randomness:** The analogy of flipping a coin or drawing a card is spot on. In situations where a deterministic choice might not be optimal, randomized algorithms can introduce a "fair chance" element to improve the overall outcome.
- **Speeding Up Brute Force:** Brute force algorithms try all possible solutions to find the best one. This can be very slow for large datasets. Randomized algorithms can sometimes help by intelligently sampling the input space, potentially finding a good enough solution much faster than trying everything.

Types of Randomized Algorithms

- *Las Vegas algorithm*
- *Monte Carlo algorithm*

Las Vegas Algorithm:

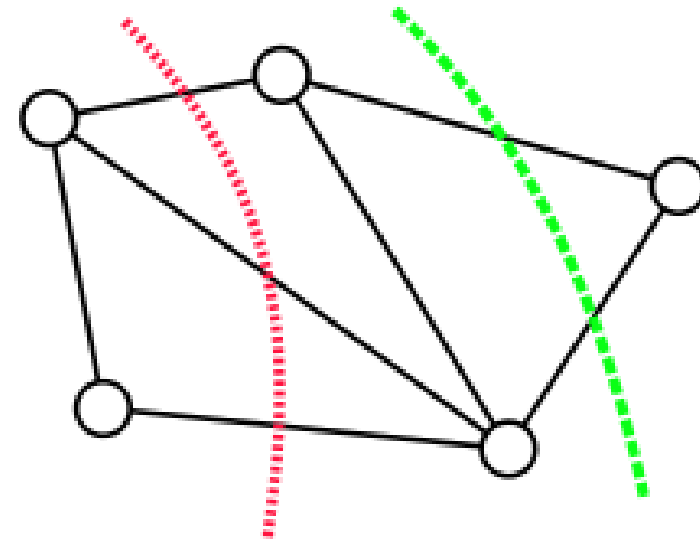
- Always produces the correct answer.
- The running time is a **random variable**, but its expected value is bounded.
- Example: **Randomized Quicksort**.
 - This algorithm always gives the correct sorted output.
 - However, its running time may vary depending on the random choices made during execution.
 - Despite the variability in running time, the expected time complexity is bounded.
- Probability bounds may be provided, such as in the case of Randomized Quicksort, where the probability that the running time exceeds a certain multiple of its expected time is very low (less than $n^{-\log \log n}$).

Types of Randomized Algorithms

- *Las Vegas algorithm:*
- *Monte Carlo algorithm:*

Monte Carlo Algorithm:

- Can produce an incorrect result with some probability.
 - The running time is deterministic.
 - **Example:** Randomized Algorithm for **Approximate Median**.
 - **Another example** is finding a minimum cut in a graph using random contractions.
-
- The algorithm may sometimes produce an incorrect minimum cut due to randomness, but it does so with a low probability.
 - Typically, these algorithms are designed to provide an acceptable result within a certain probability threshold.
 - The focus is often on ensuring a low probability of error while sacrificing some precision or optimality in the result.



Any

Question



PresenterMedia

Thank You!

**FOR YOUR
ATTENTION**

